

01__dataset__exploration

October 10, 2026

1 CardiaSense — Dataset & signal foundations

Educational technical walkthrough · notebook 1 of 5

Objectives: load the manifest, listen to the sounds, build a spectrogram, estimate quality

You will need: the `cardiasense` venv (run `./setup.sh`), the dataset at `$CARDIASENSE_DATA_DIR` (downloaded by `setup.sh`), and a trained head for the notebooks that use checkpoints (or run the smoke train in notebook 03 first).

Educational / research prototype — **not a medical device**.

1.0.1 Step 1 — Load the manifest (CSV → recordings)

The dataset's ground truth lives in `HS.csv`, `LS.csv`, `Mix.csv`. Filenames are reconstructed from the **ID columns** (never the Location column — Apex in the CSV maps to A in the filename). The manifest builder also strips whitespace (one `LS.csv` row has a leading space) and *asserts* file existence + expected counts, so a corrupt download fails loudly instead of silently.

What you should see: 535 clips for the sound task (50 heart + 50 lung + 145 mixed sessions × 3 stems), and the class balance — note how imbalanced the heart task is (89% abnormal), which is exactly why we use balanced metrics and class-weighted losses later.

```
[1]: import os, sys
from pathlib import Path

ROOT = Path("/home/tnzr/CardiaSense-AI")
sys.path.insert(0, str(ROOT / "ml"))
sys.path.insert(0, str(ROOT))

from cardia.config import DEFAULT_DATA_DIR # loads .env if present
DATA = os.environ.get("CARDIASENSE_DATA_DIR") or DEFAULT_DATA_DIR
print("Dataset root:", DATA)
```

Dataset root: /home/tnzr/CardiaSense-AI/data/HLS-CMDS

```
[2]: from cardia.data.hls_cmds import build_task_manifest
m = build_task_manifest(DATA, "sound", "binary")
print("sound:", len(m), "files")
```

```
print(m["task_label"].value_counts())
for task in ("heart", "lung"):
    mm = build_task_manifest(DATA, task, "binary")
    print(task, len(mm), "samples")
    print(mm["task_label"].value_counts().to_dict())
```

```
sound: 535 files
task_label
heart      195
lung       195
mixed      145
Name: count, dtype: int64
heart 195 samples
{'abnormal': 173, 'normal': 22}
lung 195 samples
{'abnormal': 155, 'normal': 40}
```

1.0.2 Step 2 — Listen (the most important “feature”)

Every clip is 15 s of 4 kHz, 16-bit mono audio. Listen to each type below. - **Heart:** you should hear the *lub-dub* (S1–S2); a murmur is a longer, whooshing vibration between them. - **Lung:** normal breathing is quiet; **wheezes** are high-pitched whistles, **crackles** are short pops (like hair being rubbed), **rhonchi** low rattles.

Why this matters: if you can hear it, a well-designed spectrogram + model can learn it — these clips are your sanity check for everything that follows.

```
[3]: import IPython.display as d
from cardia.data.transforms import load_wav, peak_normalize

for label in ("heart", "lung", "mixed"):
    row = m[m["task_label"] == label].iloc[0]
    wav, sr = load_wav(row["file_path"])
    display(d.Audio(wav.numpy(), rate=sr))
    print(label, row["sample_id"])
```

```
<IPython.lib.display.Audio object>
```

```
heart F_N_RC
```

```
<IPython.lib.display.Audio object>
```

```
lung M_N_RUA
```

```
<IPython.lib.display.Audio object>
```

```
mixed M0001
```

1.0.3 Step 3 — From waveform to log-mel spectrogram

A raw waveform is hard for a CNN. The standard acoustic front-end:

1. **Frame + STFT** — slice into ~100 ms windows ($n_{\text{fft}}=400$ @ 4 kHz), hop 40 ms (160 samples) $\rightarrow$ ~376 frames for 15 s.
2. **Mel filterbank** — compress frequency onto 64 mel bands (perceptual scale).
3. **Log** — $\log(S + 1e-6)$ converts the wide dynamic range of power into a perceptual “loudness” scale and stabilises training.

The result is a 64×376 image: **time on x, mel bin on y** — this is the input every deep head consumes.

```
[4]: import matplotlib.pyplot as plt
from cardia.config import Config
from cardia.data.transforms import MelSpec, load_wav, peak_normalize, preprocess
from cardia.wandb_utils import viz

cfg = Config()
row = m[m["task_label"] == "heart"].iloc[0]
raw, sr = load_wav(row["file_path"]); raw = peak_normalize(raw)
filt = preprocess(row["file_path"], cfg)           # band-pass 20-600 Hz (heart_
↳band)
mel = MelSpec(cfg.sample_rate, cfg.n_mels, cfg.n_fft, cfg.hop_length)
viz.waveform_fig(raw.numpy(), sr, f"raw {row['sample_id']}"); plt.show()
spec = mel(filt).numpy()
print("log-mel shape:", spec.shape)
viz.spec_fig(spec, f"log-mel {row['sample_id']} (64 mel bins × 376 frames)")
plt.show()
```

log-mel shape: (64, 376)

/tmp/ipykernel_558702/2600876698.py:11: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

```
viz.waveform_fig(raw.numpy(), sr, f"raw {row['sample_id']}"); plt.show()
```

/tmp/ipykernel_558702/2600876698.py:15: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

```
plt.show()
```

1.0.4 Step 4 — Signal quality (garbage in, garbage out)

`clip_ratio` (samples touching ± 1), RMS level, and an SNR estimate tell us if a recording is usable. A clipped or near-silent clip should be flagged before it is fed to a model — the web app surfaces these flags on every report.

```
[5]: from cardia.data.stats import sqi_stats
df = sqi_stats(m, n=50)
print(df[["sample_id", "duration_s", "rms", "snr_estimate", "clip_ratio"]].
↳describe().T)
```

	count	mean	std	min	25%	50%	\
duration_s	50.0	15.000000	0.000000	15.000000	15.000000	15.000000	
rms	50.0	0.098548	0.045507	0.014796	0.063869	0.089783	
snr_estimate	50.0	5.413187	1.839675	2.368986	4.139883	5.469579	

clip_ratio	50.0	0.000030	0.000019	0.000000	0.000017	0.000033
------------	------	----------	----------	----------	----------	----------

		75%	max
duration_s	15.000000	15.000000	
rms	0.128241	0.193469	
snr_estimate	6.418427	10.864118	
clip_ratio	0.000033	0.000067	

02_preprocessing_splits

October 10, 2026

1 CardiaSense — DSP, augmentation & validation discipline

Educational technical walkthrough · notebook 2 of 5

Objectives: band-pass filtering, seeded augmentation, leakage-safe grouped CV splits

You will need: the `cardiasense` venv (run `./setup.sh`), the dataset at `$CARDIASENSE_DATA_DIR` (downloaded by `setup.sh`), and a trained head for the notebooks that use checkpoints (or run the smoke train in notebook 03 first).

Educational / research prototype — **not a medical device**.

1.0.1 Step 1 — Band-pass filtering (the task-dependent “lens”)

Heart energy concentrates at **20–600 Hz**, lung sounds at **60–1500 Hz** (Nyquist = 2000 Hz at 4 kHz). We apply a zero-phase Butterworth band-pass (`scipy.signal.sosfiltfilt`) so the filter doesn’t shift the signal in time.

Technical note: `sos` (second-order sections) + `sosfiltfilt` is numerically stable and filters the signal *forwards then backwards*, removing phase lag — essential for downstream time-aligned visualisation.

```
[1]: import os, sys
      from pathlib import Path

      ROOT = Path("/home/tnzr/CardiaSense-AI")
      sys.path.insert(0, str(ROOT / "ml"))
      sys.path.insert(0, str(ROOT))

      from cardia.config import DEFAULT_DATA_DIR # loads .env if present
      DATA = os.environ.get("CARDIASENSE_DATA_DIR") or DEFAULT_DATA_DIR
      print("Dataset root:", DATA)
```

Dataset root: /home/tnzr/CardiaSense-AI/data/HLS-CMDS

```
[2]: import numpy as np
      import pandas as pd
      from cardia.config import Config
      from cardia.data.hls_cmds import build_task_manifest
```

```

from cardia.data.transforms import load_wav, peak_normalize, preprocess
from cardia.wandb_utils import viz
import matplotlib.pyplot as plt

cfg = Config()
lm = build_task_manifest(DATA, "lung", "binary")
row = lm.iloc[0]
raw, sr = load_wav(row["file_path"]); raw = peak_normalize(raw)
filt = preprocess(row["file_path"], cfg) # lung band 60-1500 Hz
viz.overlay_fig(raw.numpy(), filt.numpy(), sr, f"band-pass overlay_
↳{row['sample_id']}")
plt.show()

```

/tmp/ipykernel_558896/3704940331.py:15: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

```
plt.show()
```

1.0.2 Step 2 — Augmentation (train-only, and deterministic)

To generalise past 535 clips we synthesise plausible variants during training: **time roll**, **amplitude scaling**, **additive Gaussian noise** on the waveform, and **SpecAugment** masks on the spectrogram. Everything is driven by a seeded `torch.Generator`, re-seeded per epoch, so runs are reproducible (fold-0 reruns are bit-identical).

Watch the difference — the right panel shows the masked/perturbed version.

```

[3]: from cardia.data.transforms import Augment, MelSpec
aug = Augment(cfg, cfg.seed); aug.set_epoch(0)
w = aug.waveform(filt)
mel_spec = MelSpec(cfg.sample_rate, cfg.n_mels, cfg.n_fft, cfg.hop_length)
viz.aug_fig(mel_spec(filt).numpy(), aug.spec(mel_spec(w)).numpy(),
            f"augmentation {row['sample_id']}")
plt.show()

```

/tmp/ipykernel_558896/3539103049.py:7: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

```
plt.show()
```

1.0.3 Step 3 — Leakage-safe, grouped, stratified splits

The **critical** methodological choice: a mixed session produces H####, L#### and M#### clips that *must never* straddle train/val/test — otherwise the model “recognises” the session instead of generalising.

The splitter: 1. **Outer** StratifiedGroupKFold(5) partitions **groups** (the session index) → fold k is TEST. 2. **Inner** re-split of the remaining groups → validation (~20%). 3. **Assertions** verify train/val/test group sets are pairwise disjoint and that the test folds cover every group exactly once.

Expected shape: ~60% train / 20% val / 20% test, with every fold containing all classes (stratification).

```
[4]: from cardia.data.splits import make_splits
m = build_task_manifest(DATA, "sound", "binary")
splits = make_splits(m, n_folds=5, seed=0) # asserts group disjointness
rows = [{"fold": s.fold, "train": len(s.train), "val": len(s.val), "test":
        ↪len(s.test)} for s in splits]
print(pd.DataFrame(rows).to_string(index=False))
```

fold	train	val	test
0	321	107	107
1	321	107	107
2	321	107	107
3	321	107	107
4	321	107	107

03_benchmark_training

October 10, 2026

1 CardiaSense — Model tiers, training & the benchmark gate

Educational technical walkthrough · notebook 3 of 5

Objectives: classical baseline, deep smoke training, campaign tables and the adopt/skip gate

You will need: the `cardiasense` venv (run `./setup.sh`), the dataset at `$CARDIASENSE_DATA_DIR` (downloaded by `setup.sh`), and a trained head for the notebooks that use checkpoints (or run the smoke train in notebook 03 first).

Educational / research prototype — **not a medical device**.

1.0.1 Step 1 — Classical baseline (fast, interpretable)

Features: 13 MFCC coefficients + their Δ and Δ^2 (temporal dynamics), plus spectral centroid / bandwidth, zero-crossing rate, RMS and spectral flatness — aggregated to mean/std per clip.

Models: logistic regression (LR), random forest (RF), and a **calibrated** SVM. All use `class_weight="balanced"` to counter the imbalance. Models fit on the **train split only** (val is reserved for model selection — same protocol as the deep stages), scored on the held-out test fold.

Expected: the classical baseline is the *gate champion* on every task — a cheap, strong reference the deep tiers must beat by 1 balanced-accuracy point.

```
[1]: import os, sys
    from pathlib import Path

    ROOT = Path("/home/tnzr/CardiaSense-AI")
    sys.path.insert(0, str(ROOT / "ml"))
    sys.path.insert(0, str(ROOT))

    from cardia.config import DEFAULT_DATA_DIR # loads .env if present
    DATA = os.environ.get("CARDIASENSE_DATA_DIR") or DEFAULT_DATA_DIR
    print("Dataset root:", DATA)
```

Dataset root: /home/tnzr/CardiaSense-AI/data/HLS-CMDS

```
[2]: from cardia.config import Config
from cardia.data.hls_cmds import build_task_manifest
from cardia.data.splits import make_splits
from cardia.train import classical_train
import wandb, pandas as pd

cfg = Config(task="heart", stage=1, out_dir=str(ROOT / "ml" / "runs"))
m = build_task_manifest(DATA, "heart", "binary")
splits = make_splits(m, n_folds=5, seed=0)
run = wandb.init(mode="disabled")
metrics = classical_train.run_classical(cfg, m, splits, ["normal", "abnormal"],
    ↪run, "nb-walkthrough")
run.finish()
print(pd.DataFrame(metrics["_per_classifier"])[["classifier",
    ↪"balanced_accuracy_mean", "macro_f1_mean"]].to_string(index=False))
```

```
/home/tnzr/CardiaSense-AI/.venv/lib/python3.11/site-packages/tqdm/auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
```

	classifier	balanced_accuracy_mean	macro_f1_mean
	logistic	0.886008	0.844785
random_forest		0.882143	0.915634
	svm	0.909202	0.921345

1.0.2 Step 2 — Deep tier (smoke mode) via the CLI

The deep stages (stage 2 CNN → stage 3 temporal → stage 4 transformer) are invoked through `cardia.cli`. `--smoke` cuts to 2 folds × 2 epochs so you can see the whole path (dataloaders, class weights, cosine LR, early stopping, checkpoint, metrics) in about a minute on GPU.

What it prints: val balanced accuracy / macro-F1 per epoch, then the held-out test metrics with a confusion matrix, ECE and latency.

```
[3]: import subprocess
rc = subprocess.run([sys.executable, "-m", "cardia.cli", "--task", "heart",
    ↪"--stage", "2",
                        "--model", "resnet18", "--fold", "0", "--smoke",
    ↪"--out-dir", "/tmp/cardiasense_nb_runs"],
                    cwd=str(ROOT / "ml"))
print("CLI exit code:", rc.returncode)
```

```
wandb: [wandb.Api()] Loaded credentials for https://api.wandb.ai from
/home/tnzr/.netrc.
```

```
wandb: Currently logged in as: tnzr (tnzr-pioneer-innovations-collective) to
https://api.wandb.ai. Use `wandb login --relogin` to force relogin
```

```
wandb: Tracking run with wandb version 0.30.0
```

```

wandb: Run data is saved locally in
/tmp/cardiasense_nb_runs/wandb/wandb/run-20261010_220956-jltwiggq
wandb: Run `wandb offline` to turn off syncing.
wandb: Syncing run stage2-heart-resnet18-allfolds-s0
wandb: View project at https://wandb.ai/tnzr-pioneer-innovations-
collective/cardiasense-hls-cmds
wandb: View run at https://wandb.ai/tnzr-pioneer-innovations-
collective/cardiasense-hls-cmds/runs/jltwiggq

wandb: WARNING Artifact "stage2-heart-resnet18-allfolds-s0-fold0_best" already
exists with the same content. No new version will be created.

wandb: updating run metadata; uploading artifact run-jltwiggq-
outputpredictions-Y75LRA; uploading artifact run-jltwiggq-fold_results;
uploading artifact run-jltwiggq-cvconfusion_matrix--CF1KQ

wandb: uploading artifact run-jltwiggq-fold_results; uploading artifact run-
jltwiggq-cvconfusion_matrix--CF1KQ

wandb: uploading
media/images/cv/confusion_matrix_fig_27_ae190343e4b1b9b693d5.png; uploading
media/table/cv/confusion_matrix_28_719336bf8165f15ab7df.table.json; uploading
output.log; uploading wandb-summary.json; uploading config.yaml (+ 3 more)

wandb:
wandb: Run history:
wandb:          epoch
wandb:          lr
wandb:      train/loss
wandb:      val/accuracy
wandb:      val/auroc
wandb: val/balanced_accuracy
wandb:      val/loss
wandb:      val/macro_f1
wandb:
wandb: Run summary:
wandb:          best_epoch 0
wandb:      cv/best_epoch 0
wandb: cv/latency_ms_per_sample 5.53136
wandb:      cv/test_accuracy 0.49942
wandb:      cv/test_auroc 0.49242
wandb: cv/test_balanced_accuracy 0.5
wandb:      cv/test_ece 0.50058
wandb:      cv/test_macro_f1 0.28543
wandb: cv/test_macro_precision 0.24971
wandb:      cv/test_macro_recall 0.5
wandb:          +18 ...
wandb:
wandb: View run stage2-heart-resnet18-allfolds-s0 at: https://wandb.ai/tnzr-
pioneer-innovations-collective/cardiasense-hls-cmds/runs/jltwiggq

```

```
wandb: View project at: https://wandb.ai/tnzr-pioneer-innovations-
collective/cardiasense-hls-cmds
wandb: Synced 5 W&B file(s), 25 media file(s), 24 artifact file(s) and 0 other
file(s)
wandb: Find logs at:
/tmp/cardiasense_nb_runs/wandb/wandb/run-20261010_220956-jltwigq/logs

[cardia] task=heart manifest=195 samples
{
  "task": "heart",
  "stage": 2,
  "model": "resnet18",
  "fold": null,
  "seed": 0,
  "test_accuracy_mean": 0.49942141805175677,
  "test_accuracy_std": 0.547550285747336,
  "test_balanced_accuracy_mean": 0.5,
  "test_balanced_accuracy_std": 0.0,
  "test_macro_f1_mean": 0.2854313931919587,
  "test_macro_f1_std": 0.26094214750660133,
  "test_macro_precision_mean": 0.24971070902587839,
  "test_macro_precision_std": 0.273775142873668,
  "test_macro_recall_mean": 0.5,
  "test_macro_recall_std": 0.0,
  "test_auroc": 0.49242424242424243,
  "test_ece_mean": 0.5005785819482432,
  "test_ece_std": 0.547550285747336,
  "val_balanced_accuracy_mean": 0.5,
  "val_balanced_accuracy_std": 0.0,
  "latency_ms_per_sample": 5.531359631972719,
  "best_epoch": 0
}

CLI exit code: 0
```

1.0.3 Step 3 — The campaign & the gate rule

`scripts/run_sequential.sh` runs all four stages for every task, then applies the **gate**: a stage is adopted only if balanced accuracy **or** macro-F1 improves by 1 point over the best prior stage. Both metrics are logged, so the decision is auditable in `ml/runs/metrics.json` and the benchmark table.

```
[4]: import json
p = ROOT / "ml" / "runs" / "benchmark.json"
if p.exists():
    bench = json.loads(p.read_text())["benchmark"]
    print(pd.DataFrame(bench).to_string(index=False))
else:
```

```
print("No benchmark.json yet - run: cd ml && SMOKE=1 ./scripts/
↳run_sequential.sh")
```

task	stage	best_test_balanced_accuracy	best_test_macro_f1	gate_decision
sound	1	0.934159	0.935068	adopt
NaN		NaN		
sound	2	0.938167	0.939672	skip
0.934159		0.935068		
sound	3	0.851105	0.856311	skip
0.934159		0.935068		
sound	4	0.860242	0.863494	skip
0.934159		0.935068		
heart	1	0.909202	0.921345	adopt
NaN		NaN		
heart	2	0.652521	0.535970	skip
0.909202		0.921345		
heart	3	0.668235	0.454096	skip
0.909202		0.921345		
heart	4	0.464706	0.449066	skip
0.909202		0.921345		
lung	1	0.968548	0.974335	adopt
NaN		NaN		
lung	2	0.727823	0.744358	skip
0.968548		0.974335		
lung	3	0.639919	0.610161	skip
0.968548		0.974335		
lung	4	0.858065	0.841928	skip
0.968548		0.974335		

04_report_embeddings_webapp

October 10, 2026

1 CardiaSense — Embeddings, product report & web app

Educational technical walkthrough · notebook 4 of 5

Objectives: encoder token embeddings, ensemble report internals, README audio/screenshots

You will need: the `cardiasense` venv (run `./setup.sh`), the dataset at `$CARDIASENSE_DATA_DIR` (downloaded by `setup.sh`), and a trained head for the notebooks that use checkpoints (or run the smoke train in notebook 03 first).

Educational / research prototype — **not a medical device**.

1.0.1 Step 1 — The encoder as a tokenizer (research direction A)

The AST-style transformer splits the spectrogram into **patches** and maps each to a token; a `[CLS]` token pools the whole clip. `encode()` exposes these internals: `(patch_tokens (T, d), cls (d))`.

Hypothesis: the encoder already *organises* these vectors so class embeddings cluster — measure it below with the **cosine similarity between class-mean CLS embeddings**. A sharp diagonal = classes separated in embedding space → those vectors are a ready-made, ordered interface for a downstream LLM (frozen encoder + linear projection, or patch tokens as soft prompts).

If no transformer checkpoint is present this cell prints a note instead of failing — run 03's full campaign first to have one.

```
[1]: import os, sys
    from pathlib import Path

    ROOT = Path("/home/tnzr/CardiaSense-AI")
    sys.path.insert(0, str(ROOT / "ml"))
    sys.path.insert(0, str(ROOT))

    from cardia.config import DEFAULT_DATA_DIR # loads .env if present
    DATA = os.environ.get("CARDIASENSE_DATA_DIR") or DEFAULT_DATA_DIR
    print("Dataset root:", DATA)
```

Dataset root: /home/tnzr/CardiaSense-AI/data/HLS-CMDS

```
[2]: import matplotlib.pyplot as plt
from cardia.config import HEART_TYPES

MC = "/media/tnzr/AuxVolume/cardiasense-runs/mc" # multiclass campaign
↳(Transformer heads)
try:
    from cardia.features import embeddings as emb
    data = emb.corpus_embeddings(DATA, MC)
    M = emb.class_mean_matrix(data)
    print("clips:", len(data["rows"]["sample_id"]), "| embedding dim:",
↳data["rows"]["cls"][0].shape)
    fig, ax = plt.subplots(figsize=(8, 7))
    ax.imshow(M, cmap="magma", vmin=0, vmax=1)
    ax.set_xticks(range(len(HEART_TYPES))); ax.
↳set_yticks(range(len(HEART_TYPES)))
    ax.set_xticklabels([c[:10] for c in HEART_TYPES], rotation=45, ha="right")
    ax.set_yticklabels([c[:10] for c in HEART_TYPES])
    ax.set_title("Class-mean CLS embedding cosine similarity
↳(encoder-organised)")
    plt.show()
except FileNotFoundError as e:
    print("Transformer checkpoint not available here:", e)
```

Transformer checkpoint not available here: [Errno 2] No such file or directory:
'/media/tnzr/AuxVolume/cardiasense-runs/mc/splits/heart_10class_s0/fold0_train.csv'

1.0.2 Step 2 — A product-style report in one call

The web app runs **5-fold ensembles** of the heads, sweeps the clip in a sliding window, and renders a **synchronized inference timeline** (spectrogram on top, stacked-area probabilities in the middle, confidence line at the bottom — all time-aligned). This cell produces the same objects the report page renders, so you can inspect the structure directly.

```
[3]: import sys, torch, soundfile as sf
sys.path.insert(0, str(ROOT / "webapp"))
from webapp.app import inference, report, game

reg = inference.get_registry() # warm-loads head ensembles (CPU
↳here)
opts = report.ReportOptions(heads=["source", "heart_types"], figures="all",
temporal=True, temporal_mode="multiscale",
↳explanation="standard")
path = game.resolve_audio_path("M0001", "sound", "binary")
wav, sr = sf.read(path, dtype="float32", always_2d=True)
rep = report.analyze_waveform(torch.from_numpy(wav[:, 0]).squeeze(), int(sr),
↳"M0001.wav", opts)
```

```
print("verdict:", rep["verdict"], "| heads used:", [h["key"] for h in_
↳rep["heads"]])
print("figures:", list(rep["figures"].keys())[:6], "...")
```

```
verdict: no-flag | heads used: ['source']
figures: ['waveform', 'overlay', 'spectrogram', 'probs_source',
'multiscale_source', 'timeline_source'] ...
```

1.0.3 Step 3 — What ships in the repo

The GitHub README plays sample clips directly (HTML5 <audio> — GitHub renders them from committed WAVs) and the walkthrough embeds web-app screenshots. The commands below point you at those artifacts; `scripts/screenshot_walkthrough.py` regenerates the screenshots and `scripts/make_audio_assets.py` the clips.

```
[4]: print("Audio assets:  docs/assets/audio/*.wav    (played inline in the README)")
      print("Screenshots:  docs/walkthrough/*.png    (instructional walkthrough)")
      print("Report:       ml/report/cardiasense_report.pdf (regenerate via cardia.
↳report.report)")
```

```
Audio assets:  docs/assets/audio/*.wav    (played inline in the README)
Screenshots:   docs/walkthrough/*.png    (instructional walkthrough)
Report:        ml/report/cardiasense_report.pdf (regenerate via
cardia.report.report)
```

05_llm_integration

October 10, 2026

1 CardiaSense — LLM integration & forward architecture

Educational technical walkthrough · notebook 5 of 5

Objectives: provider/synthesizer narrative, personalization, chat, and two research directions

You will need: the `cardiasense` venv (run `./setup.sh`), the dataset at `$CARDIASENSE_DATA_DIR` (downloaded by `setup.sh`), and a trained head for the notebooks that use checkpoints (or run the smoke train in notebook 03 first).

Educational / research prototype — **not a medical device**.

1.0.1 Step 1 — From probabilities to a personalized narrative

The heads output probabilities — a patient wants *words*. The LLM integration (`webapp/app/llm.py`) has two interchangeable providers that return the **same JSON shape**:

- `LLMClient` — calls an OpenAI-compatible endpoint (`CARDIASENSE_LLM_*`), asked for strict JSON, grounded in the findings + patient context.
- `LLMSynthesizer` — an **offline rule-based fallback** built from a per-class knowledge base (medical / diet / environment outlines + context hints), so the product works with no API key at all.

Below we synthesize from a plausible finding vector and a short patient note. Notice the **personalization** (context hints folded into the medical recommendations) and the guardrail: every output carries the disclaimer.

```
[1]: import os, sys
from pathlib import Path

ROOT = Path("/home/tnzr/CardiaSense-AI")
sys.path.insert(0, str(ROOT / "ml"))
sys.path.insert(0, str(ROOT))

from cardia.config import DEFAULT_DATA_DIR # loads .env if present
DATA = os.environ.get("CARDIASENSE_DATA_DIR") or DEFAULT_DATA_DIR
print("Dataset root:", DATA)
```

Dataset root: /home/tnzr/CardiaSense-AI/data/HLS-CMDS

```
[2]: from webapp.app import llm as llm_mod
import json

findings = [
    {"key": "heart_binary", "label": "Heart screening", "kind": "binary", "pred": "abnormal",
     "confidence": 0.82, "classes": ["normal", "abnormal"], "probs": [0.18, 0.82]},
    {"key": "lung_types", "label": "Lung typing", "kind": "multiclass", "pred": "Wheezing",
     "confidence": 0.71, "classes": ["Normal", "Wheezing", "Fine Crackles", "Coarse Crackles",
                                     "Rhonchi", "Pleural Rub"]},
]
out = llm_mod.generate_narrative(findings, patient_context="58y, asthma, mild palpitations")
print("provider:", out["provider"], "| model:", out["model"], "| personalized:",
      out["personalized"])
print("\nSUMMARY:", out["summary"])
print("\nFINDINGS:"); [print(" -", f) for f in out["findings"]]
for area, recs in out["recommendations"].items():
    print(f"\n[{area}]"); [print(" -", r) for r in recs]
```

provider: synthesizer | model: rule-based-synthesizer-v1 | personalized: True

SUMMARY: The model flagged at least one possible abnormality. A clinician should interpret these findings with your full history and an examination.

FINDINGS:

- Heart screening: abnormal (confidence 82%)
- Lung typing: Wheezing (confidence 71%)

[medical]

- Personalized from your stated context: Patient context mentions asthma - airway findings should be weighed accordingly.; Patient reports palpitations - correlate with rhythm findings and ECG.
- Evaluate for asthma/COPD with a clinician; use reliever inhalers as prescribed

[diet]

- Avoid known food triggers if asthma is food-sensitive
- Stay hydrated to keep airways moist

[environment]

- Reduce smoke, dust and strong fumes
- Keep indoor humidity moderate

1.0.2 Step 2 — Follow-up conversation (chat semantics)

The report page's chat sends `{findings, patient_context, history}` to `POST /api/llm/report`; the LLM (or synthesizer) answers with the same shape, so follow-ups inherit the auscultation context. With a key set, the live model handles the dialogue; without one, the synthesizer keeps the flow working.

```
[3]: if os.environ.get("CARDIASENSE_LLM_API_KEY"):
    narr = llm_mod.generate_narrative(findings, "asthma",
                                     [{"role": "user", "content": "should I_
↪worry about the wheeze?"]])
    print("provider:", narr["provider"], "|", narr.get("summary", "")[:160])
else:
    print("No CARDIASENSE_LLM_API_KEY - using the offline synthesizer (shown_
↪above). "
          "Set it (or a local Ollama base URL) to exercise the live LLM path.")
```

No CARDIASENSE_LLM_API_KEY - using the offline synthesizer (shown above). Set it (or a local Ollama base URL) to exercise the live LLM path.

1.0.3 Step 3 — Two research directions worth your time

1. **Token/embedding architecture (done in part, notebook 04).** The encoder is already a tokenizer: reuse its patch/CLS embeddings as the audio interface to a language model (frozen encoder + linear projection, or patch tokens as soft prompts). The served Transformer is ~0.34 GFLOPs / 2 MB int8 — cheap enough to run on-device and let the cloud LLM do the talking.
2. **Whole-breathing-cycle modeling.** A 1–3 s snippet can miss the pattern; proper auscultation wants the whole cycle (the app's macro pass already uses the full 15 s clip). Open paths: envelope-based **cycle segmentation** with per-cycle embeddings + cross-cycle attention, **long-context encoders** for minute-long exams, and **cycle-aware training** to reduce short-clip label noise. Full design + evaluation plan in `docs/LLM_Integration.md`.